

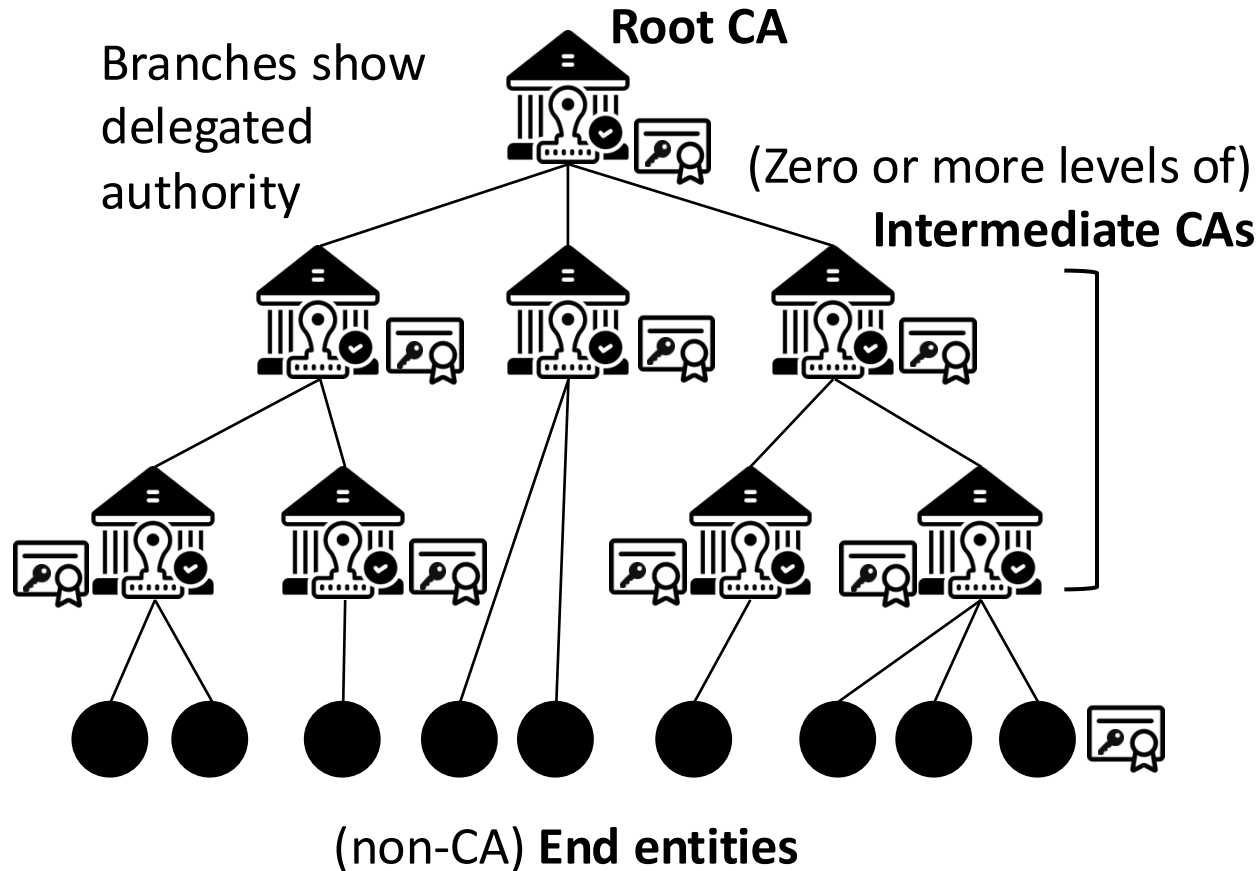
Public Key Infrastructure

– Certificate Chain and Validation

Cloud Security

CSE4059

Root and Intermediate CAs



- **Root CA**

- The top-level authority in the hierarchy
- Its trust information is configured independently at the client

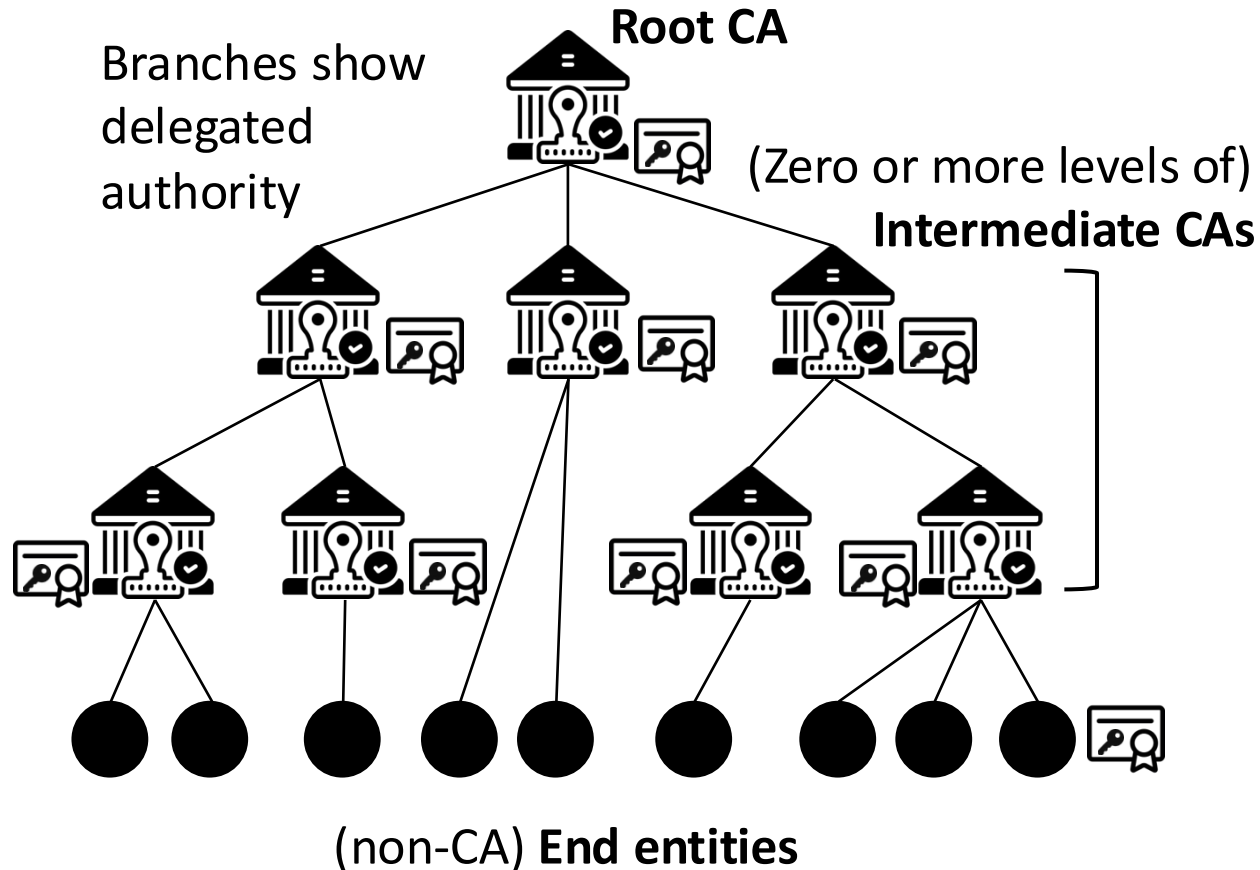
- **Intermediate CA**

- Receives a certificate from a higher CA
- It is authorized by a higher-level CA to issue certificates

- **End entity**

- ex) a domain
- Uses a leaf certificate issued by a CA

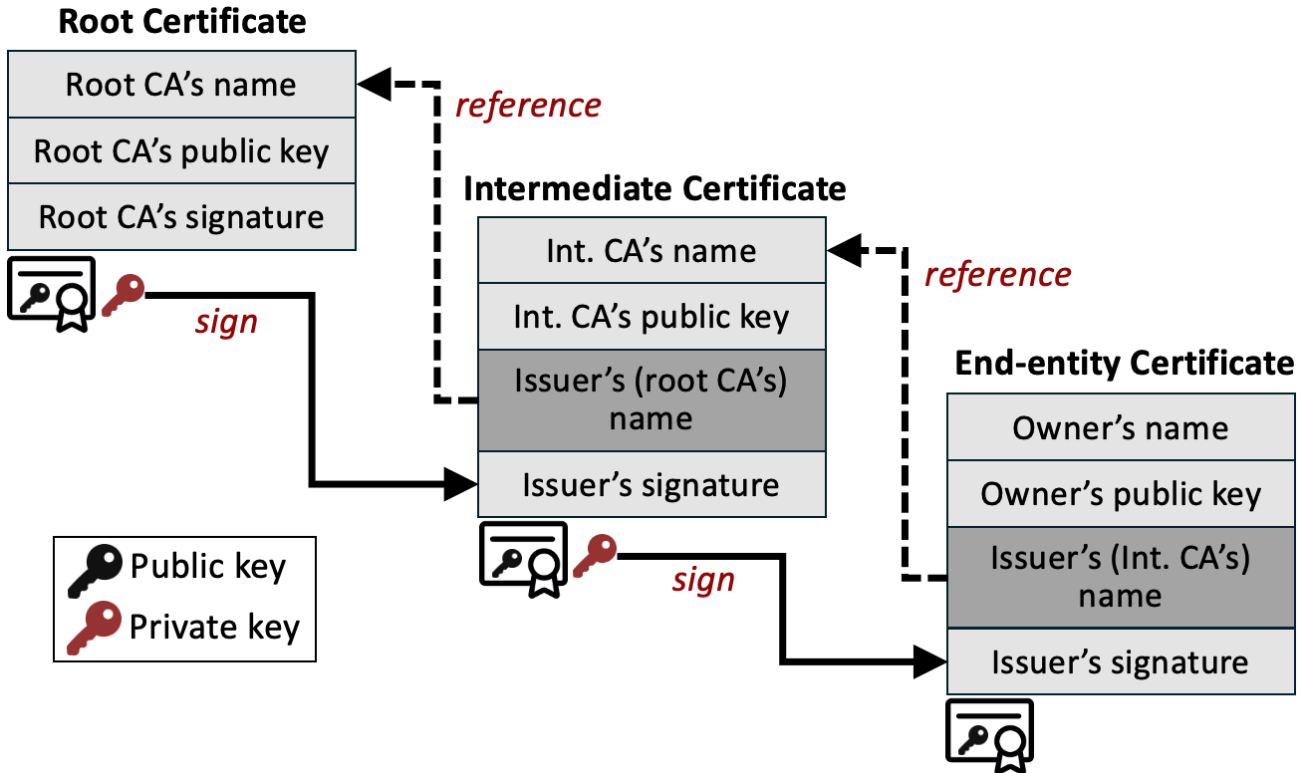
Root and Intermediate CAs (Cont'd)



Why use a hierarchy?

- Without a hierarchy (e.g., flat)
 - Each issuing CA requires a direct trust relationship with clients
- With a hierarchy
 - Clients trust root CAs rather than every issuing CA
 - Intermediate CAs can be added or replaced without updating every client's trust configuration
 - Routine certificate issuance can be delegated to intermediate CAs → Root CA's key usage can be minimized
 - Different issuance policies and purposes can be separated

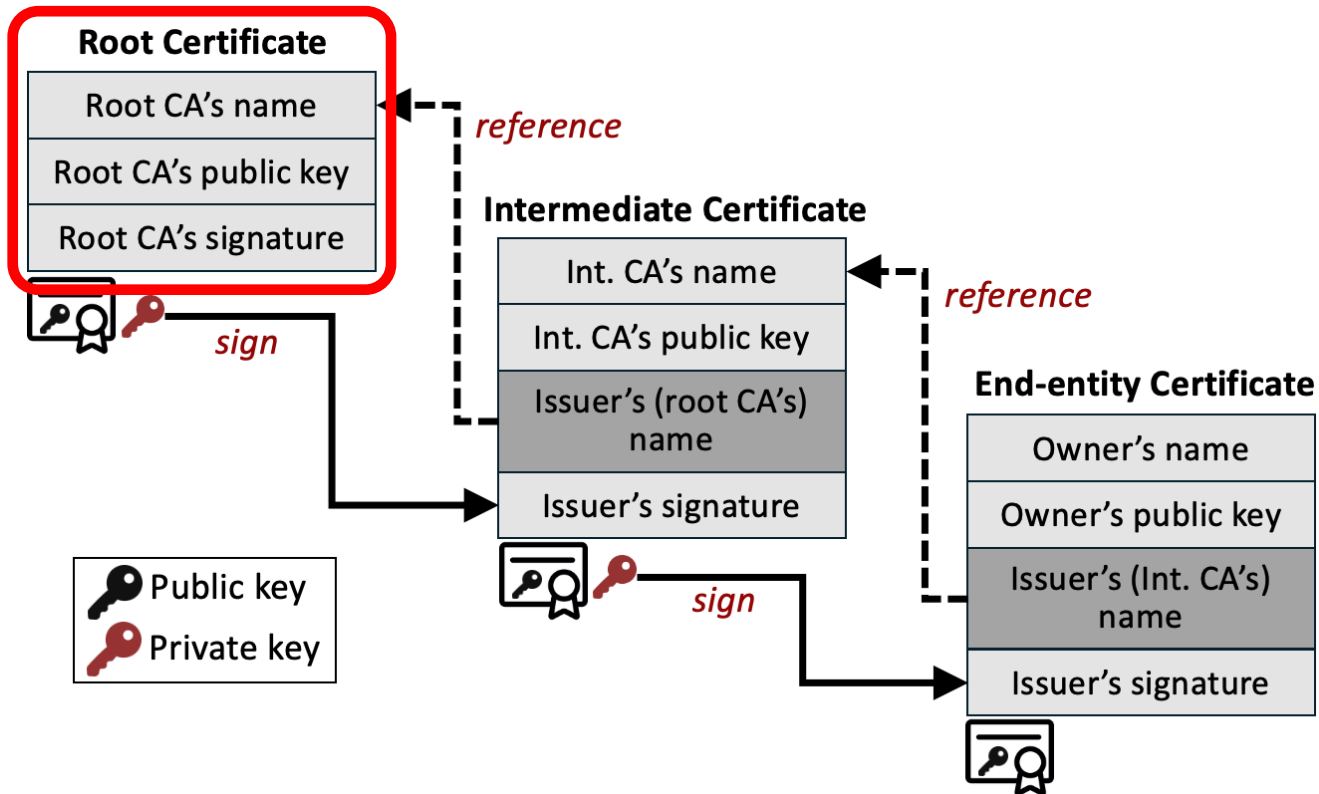
Certificate Chain



How certificates are connected

- Each (non-root) certificate contains information about its issuer
 - A leaf certificate refers to its issuer CA
 - An intermediate certificate refers to a higher-level CA (e.g., root)
- Each issuer signs the certificate below it
 - Root CA signs the intermediate CA certificate
 - Intermediate CA signs the leaf certificate

Self-Signed Certificate



- A self-signed certificate is signed with its subject's own private key
 - Its own public key verifies the self-signature
- A valid self-signature does not establish trust
 - Root certificates are usually self-signed
 - Their trust comes from independent configuration

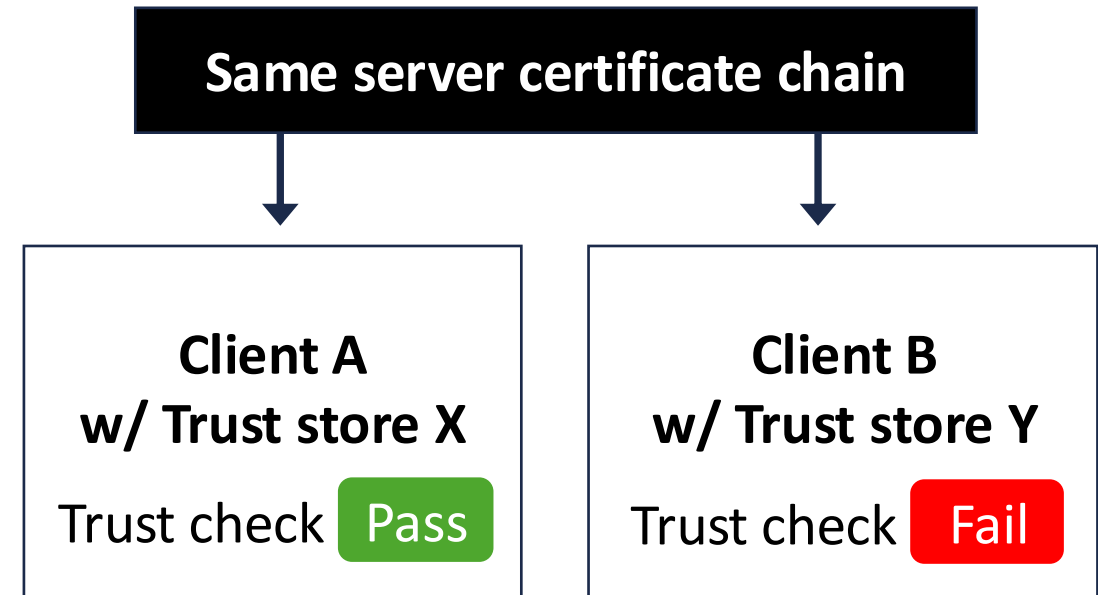
Trust Stores

- A trust store records the CAs a client accepts as trust anchors
 - Usually contains trusted root CA certificates
- The Operating System (OS), browser, or application may maintain its own store
 - Ex) Ubuntu CA certificate bundle: '/etc/ssl/certs/ca-certificates.crt'
 - Ex) Chrome Root Store →

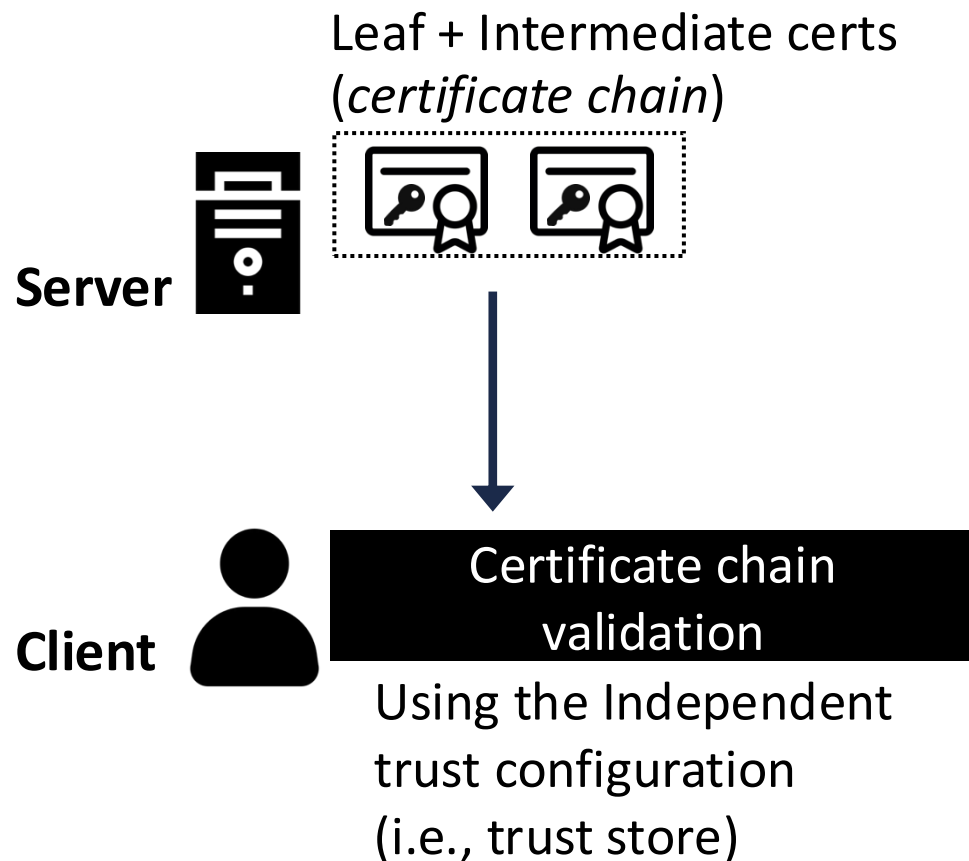
Chrome Root Store			
Version: 39			
Contents			
• Trusted Roots • Constrained Roots			
Trusted Roots			
SHA 256 Hash	Subject	NotBefore	NotAfter
55926084ec963a64b96e2abe01ce0ba86a64fbfebcc7aab5afc155b37fd76066	CN=Actalis Authentication Root CA,O=Actalis S.p.A./03358520967,L=Milan,C=IT	2011-09-22	2030-09-22
8ecde6884f3d87b1125ba31ac3fcb13d7016de7f57cc904fe1cb97c6ae98196e	CN=Amazon Root CA 1,O=Amazon,C=US	2015-05-26	2038-01-17
e35d28419ed02025cfa69038cd623962458da5c695fbdea3c22b0bfb25897092	CN=Amazon Root CA 4,O=Amazon,C=US	2015-05-26	2040-05-26
657cfe2fa73faa38462571f332a2363a46fce7020951710702cdfbb6eeda3305	OU=certSIGN ROOT CA G2,O=CERTSIGN SA,C=RO	2017-02-06	2042-02-06
5cc3d78e4e1d5e45547a04e6873e64f90cf9536d1ccc2ef800f355c4c5fd70fd	CN=CFCA EV ROOT,O=China Financial Certification Authority,C=CN	2012-08-08	2029-12-31
e23d4a036d7b70e9f595b1422079d2b91edfbb1fb651a0633eaa8a9dc5f80703	CN=CA Disig Root R2,O=Disig a.s.,L=Bratislava,C=SK	2012-07-19	2042-07-19

Trust Stores (Cont'd)

- A trust store records the CAs a client accepts as trust anchors
 - Usually contains trusted root CA certificates
- The Operating System (OS), browser, or application may maintain its own store
- The same chain can be accepted by one client and rejected by another
 - Trust depends not only the certificate chain from the server, but also on the client's own trust configuration



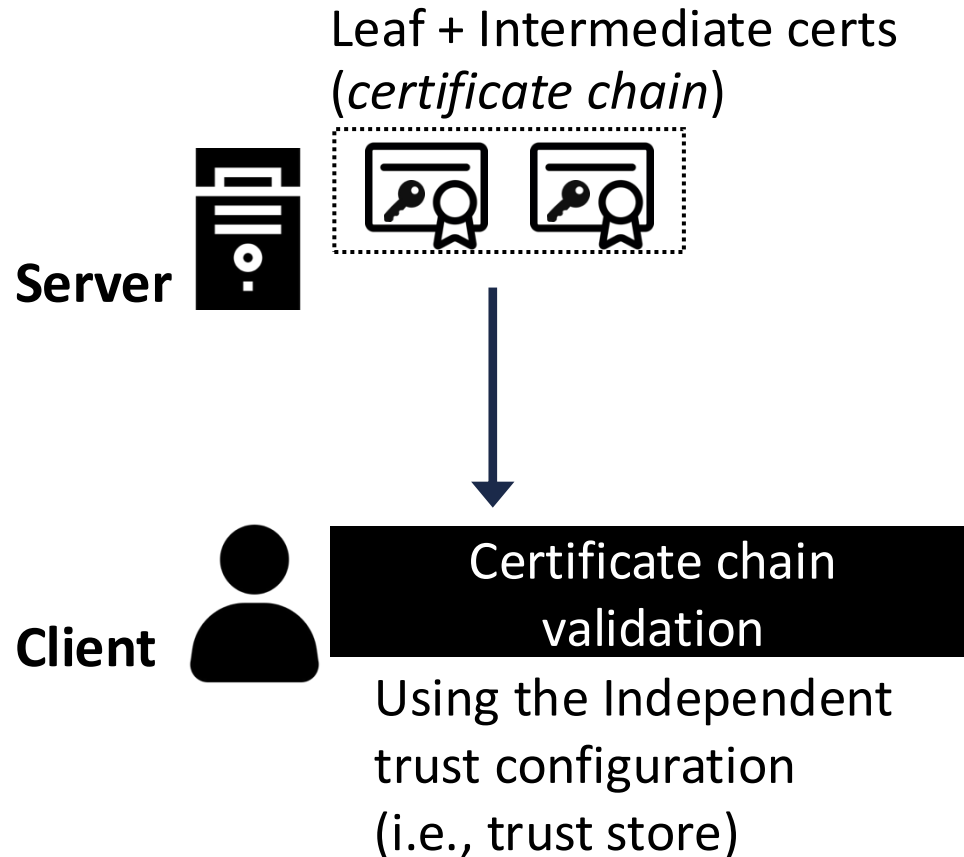
Certificate Distribution and Validation



- **Chain distribution**

- The server typically sends its leaf certificate and needed intermediate certificates
- These certificates allow the client to connect the leaf certificate toward a trusted root

Certificate Distribution and Validation



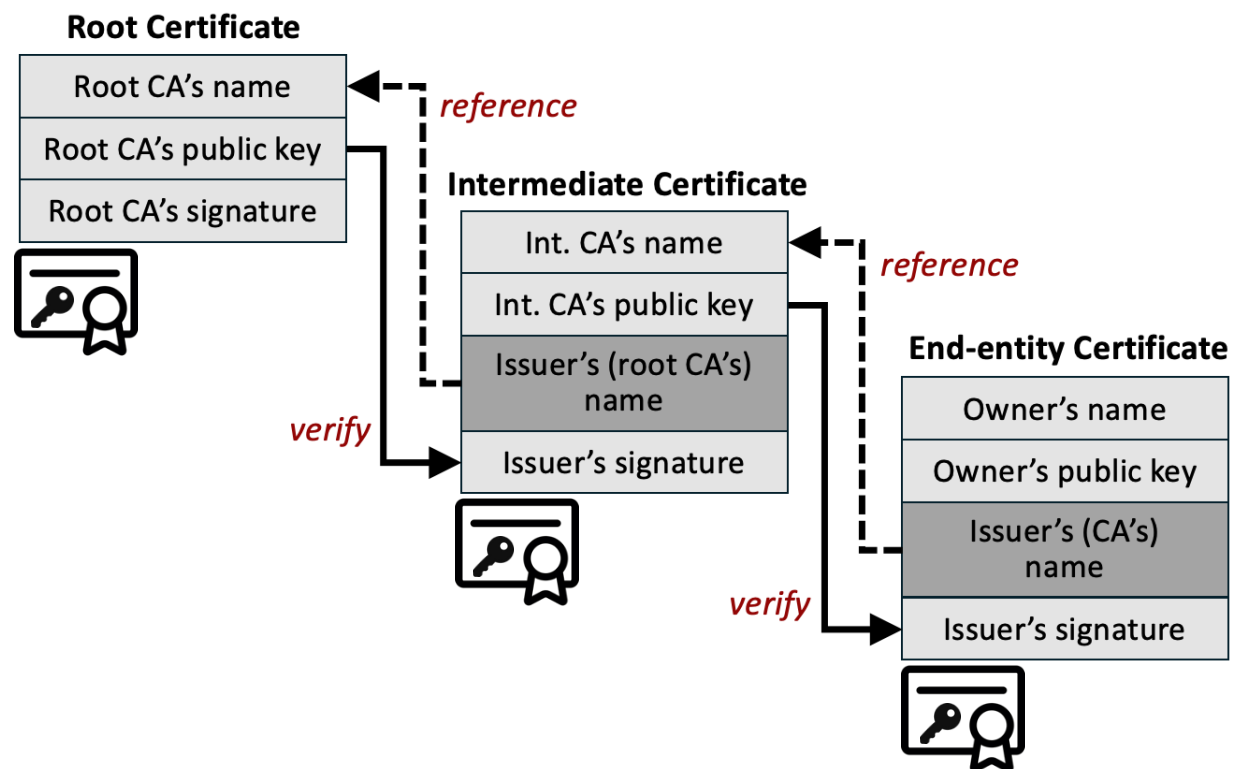
- **Chain distribution**

- The server typically sends its leaf certificate and needed intermediate certificates
- These certificates allow the client to connect the leaf certificate toward a trusted root

- **Certificate chain validation**

- The client checks
 - Chain reaches an accepted trust anchor; signatures verify
 - Relevant certificates are within their validity periods

Certificate Chain Validation



- To validate the chain, the client works upward from the leaf certificate
1. Find the issuer certificate
 2. Verify the lower certificate's signature using the issuer's public key
 3. Repeat for each certificate in the chain
 4. Eventually reach a root CA that the client already trusts

Validity Period Check

- Check the leaf and intermediate certificates at the same validation time
 - Each has a 'notBefore – notAfter' validity interval
- The validity of each certificate should be examined *individually*

- Example) Validation on October 1

- Leaf cert: valid
- Intermediate cert: Expired

→ The certificate chain fails validation

- A valid leaf cannot compensate for an expired intermediate

Certificate	Validity (in 2026)
Intermediate	January 1 – September 30
Leaf	September 1 – November 30

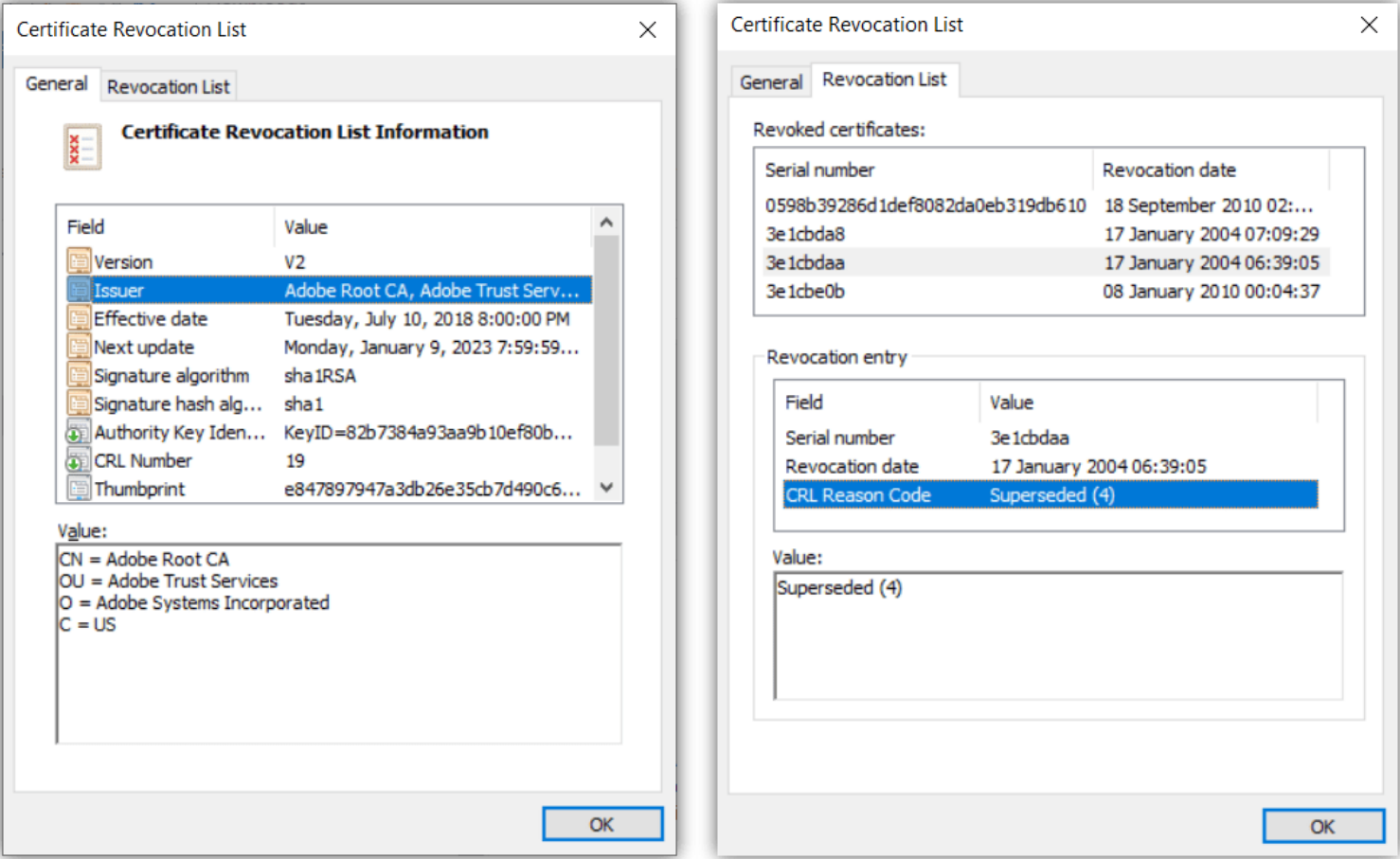
Certificate Revocation

- **Revocation** withdraws an issued certificate *before its expiry date*
- Why?
 - Private key compromise or suspected compromise
 - Certificate mis-issuance
 - ...
- An issued certificate cannot simply be modified
 - Its validity period and other fields are protected by the issuer's signature
- Replacing the certificate does not tell clients to reject the old one
- We need a mechanism that can say “Do not trust this certificate anymore”

Revocation Mechanism: CRL

- Certificate Revocation Lists (CRLs)
 - A signed list of certificates revoked by a CA before expiration
 - It is like a blacklist of certificate serial numbers
 - Periodically reissued according to the CA's policy (e.g., daily)
 - The CRL Distribution Points extension in a certificate tells clients where to retrieve the CRL (ex. URL)
- A CRL contains
 - Issuer
 - thisUpdate – when the CRL was generated
 - nextUpdate – when the next CRL will be published
 - Revoked certificate entries
 - Serial number, revocation date, ...
 - Signature & signature algorithm

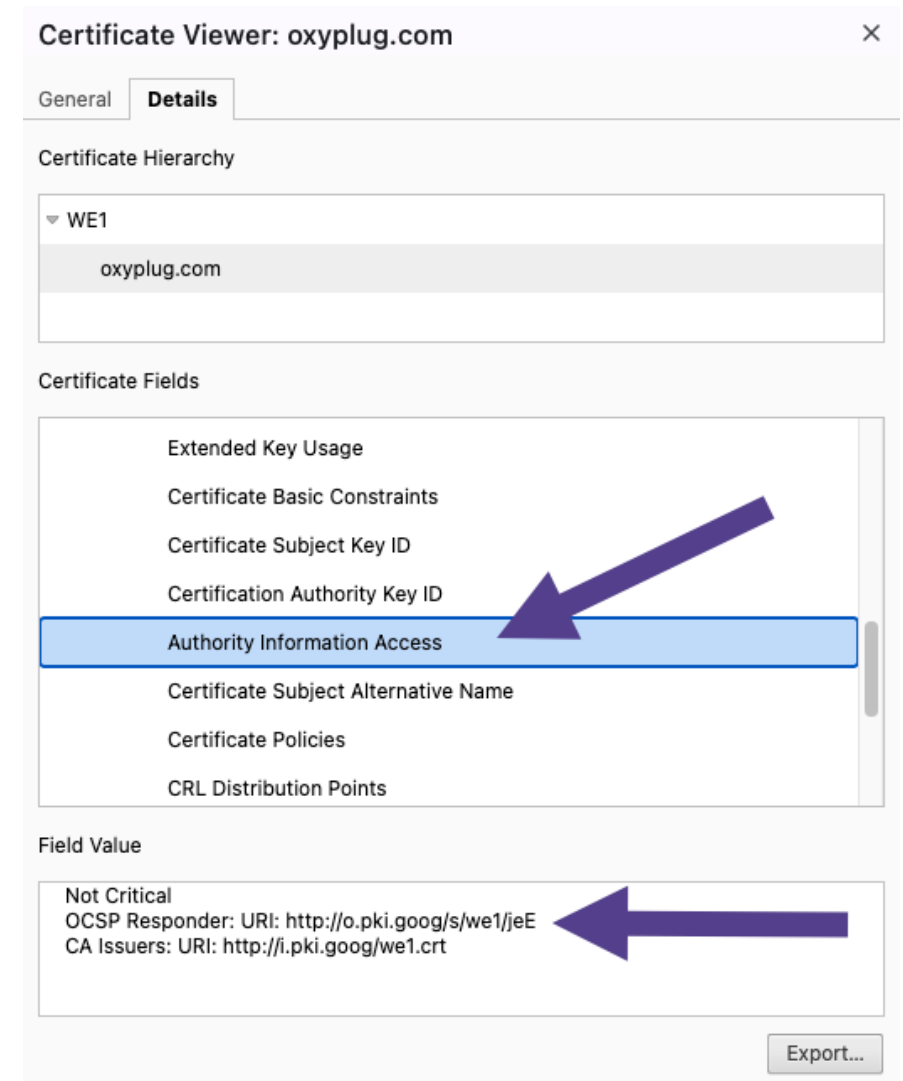
Revocation Mechanism: CRL



Source: <https://sectigostore.com/blog/what-is-a-crl-a-certificate-revocation-list-explained/>

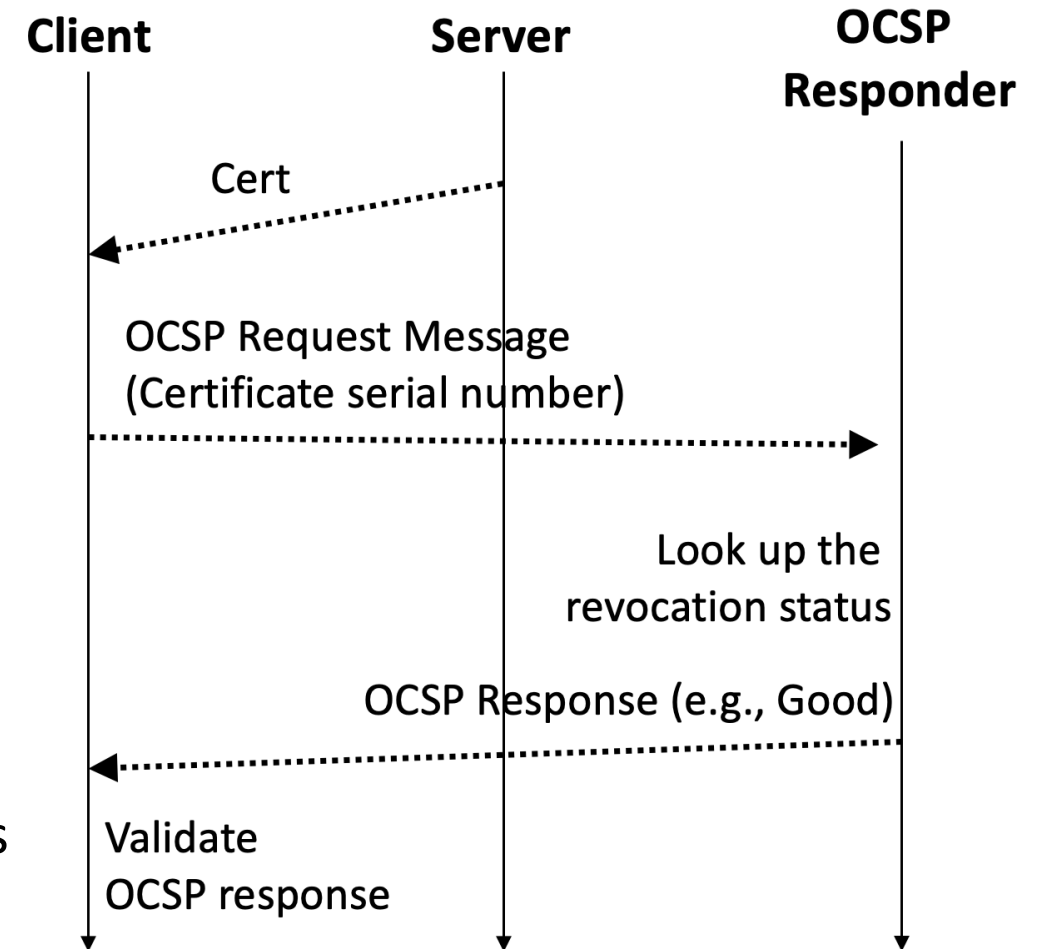
Revocation Mechanism: OCSP

- Online Certificate Status Protocol (OCSP)
 - A protocol for checking the revocation status of a specific certificate
 - Instead of downloading an entire CRL, the client queries an OCSP responder
- OCSP responder
 - Operated by or authorized by the CA
 - The certificate usually provides the responder location in the Authority Information Access (AIA) extension (e.g., URL)



Revocation Mechanism: OCSP

- How OCSP works
 - A client sends a request identifying the certificate
 - Issuer information + certificate serial number
 - OCSP responder returns a signed response
 - Good, revoked, unknown
 - The client checks
 - The response signature (e.g., generated by the responder)
 - The response corresponds to the requested certificate
 - The response is recent enough (e.g., thisUpdate/nextUpdate)
- OCSP 'Good' does not replace other certificate checks



Revocation Mechanisms: CRL and OCSP

- CRL vs. OCSP

Client action	CRL	OCSP
Obtain evidence	Retrieve a signed revocation list	Query an OCSP responder for a specific certificate
Interpret evidence	Check whether the certificate's serial number appears in the applicable CRL	Read the signed status: good, revoked, or unknown

Validation Failures Summary

- Different validation failures have different causes

Failure	Reason
Invalid signature	Signature cannot be validated
Missing intermediate	Cannot build a valid chain
Expired certificate	Validity period checks fail
Revoked certificate	Revocation status indicates that the certificate has been revoked

PKI's Problems

- Any CA can issue a certificate for any named entity
- Compromise or misbehavior of one trusted CA can have broad impact
- If a CA is compromised, its recovery is a very painful process
 - e.g., revoking/reissuing its certificates
- Verifying certificates is dependent on the implementations of browsers and web servers

CA Misissuance

- Misissuance occurs when a CA issues a certificate without proper authorization
 - ex) a CA incorrectly issues a certificate that binds *example.com* to an attacker-controlled public key, even though the attacker does not control *example.com*
- The certificate can still have
 - A valid CA signature
 - A valid chain to a trusted root
- Client-side validation may therefore succeed
- PKI depends on correct CA operation, not only cryptographic verification

CA Misissuance

- A long history of CAs getting hacked or doing the wrong thing
 - 2011: Comodo and DigiNotar CAs hacked (separately)
 - Used to issue fraudulent certificates for services including Gmail, Skype, Firefox...
 - 2013: TurkTrust issued fraudulent certificate
 - 2014: Indian NIC issued certs for Google and Yahoo!
 - 2015: CNNIC issued certs for Google domains
 - 2016: WoSign issues cert for GitHub
 - 2017: Symantec issued certs without authorization

“Only as secure as its weakest link (e.g., CA)”

Mitigations

- Certificate pinning
- Certificate Transparency (CT)

Mitigations – Certificate Pinning

- Pinning restricts which certificate (or public key) a client will accept for a service
- A client stores an expected certificate or public key
 - A normally valid certificate from another CA can then be rejected if it does not match the pin
 - This can reduce the impact of CA misissuance
- But pinning creates operational risks
 - Ex) during key rotation or certificate changes
- Browser-based pinning is largely a historical mechanism
 - Pinning is still used in some applications and controlled environments

Mitigations – Certificate Transparency

- Certificate Transparency (CT) provides public, auditable records of certificate issuance
- CT logs record certificates
 - or pre-certificates – signed statements of intended certificate issuance
- Domain owners or monitoring services can inspect the logs for unexpected certificates
 - Makes unexpected certificate issuance visible to monitors
- CT improves detection and accountability
 - It does not prevent misissuance by itself

Summary

- Certificate chains connect end-entity certificates to trusted CA authority
 - Root CA
 - Intermediate CAs
 - End-entity
- Certificate chain validation checks
 - Trust anchor, certificate signatures, validity periods
- CA misissuance remains possible